BÀI THỰC HÀNH – PROCESSES IN LINUX

Advanced Programming Techniques – Lecture 3

Mục tiêu: Bài thực hành giúp sinh viên quan sát trực tiếp cách process được tạo, thực thi, đồng bộ và trao đổi dữ liệu trong Linux.

Quy ước thực hành

- Mỗi bài được lưu trong một file .c riêng.
- Biên dịch theo dạng: gcc ten_file.c -o ten_chuong_trinh
- Chạy chương trình, quan sát kết quả và trả lời ngắn các câu hỏi đi kèm.

Bài 1. Quan sát một process đang chạy

Tạo chương trình in PID, PPID và giữ process tồn tại trong 20 giây:

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("PID  = %d\n", getpid());
    printf("PPID = %d\n", getppid());

    sleep(20);
    return 0;
}
```

Biên dịch và chạy:

```
gcc process_info.c -o process_info
./process_info
```

Trong một Terminal khác, dùng:

```
ps -o pid,ppid,state,cmd
```

Yêu cầu:

- Tìm process vừa chạy trong kết quả của ps.
- Đối chiếu PID và PPID với kết quả chương trình.
- Giải thích ngắn: file thực thi process_info và process đang chạy khác nhau ở điểm nào?

Bài 2. Tạo process bằng fork()

Hoàn thiện chương trình để phân biệt parent và child:

```
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child : PID=%d, PPID=%d\n", getpid(), getppid());
    } else {
        printf("Parent: PID=%d, Child PID=%d\n", getpid(), pid);
    }
```

```
    return 0;
}
```

Thực hiện chương trình ít nhất 5 lần và quan sát thứ tự hai dòng kết quả.

Yêu cầu:

- Nêu giá trị fork() trả về trong parent và child.
- Giải thích vì sao không thể khẳng định parent luôn in trước child.

Bài 3. Bộ nhớ của parent và child sau fork()

Cho biến:

```
int x = 10;
```

Sau fork(), cho parent gán x = 20 và child gán x = 30. Mỗi process in PID và giá trị x của mình.

Yêu cầu:

- So sánh giá trị x ở parent và child.
- Giải thích kết quả dựa trên khái niệm private virtual address space.

Bài 4. Kết hợp fork(), exec() và waitpid()

Viết chương trình thực hiện:

- Parent tạo một child bằng fork().
- Child dùng execlp() để chạy lệnh ls -l.
- Parent dùng waitpid() để chờ child kết thúc, sau đó in "Child finished".

Các lệnh cần sử dụng:

```
execlp("ls", "ls", "-l", NULL);
waitpid(child_pid, NULL, 0);
```

Yêu cầu:

- Giải thích fork() và exec() khác nhau như thế nào.
- Thêm một printf() ngay sau execlp() và kiểm tra xem dòng đó có chạy khi exec() thành công hay không.
- Giải thích vai trò của waitpid().

Bài 5. Xử lý SIGINT

Viết chương trình đếm tăng dần mỗi giây. Đăng ký một signal handler cho SIGINT.

Khi nhấn Ctrl+C, chương trình phải in:

```
SIGINT received
```

sau đó tiếp tục chạy thay vì kết thúc ngay.

Yêu cầu:

- Xác định signal được tạo ra khi nhấn Ctrl+C.
- Phân biệt default action và signal handler.

Bài 6. Truyền dữ liệu từ parent sang child bằng pipe

Viết chương trình theo mô hình:

```
Parent  --write-->  pipe  --read-->  Child
```

Yêu cầu chương trình:

- Tạo pipe trước khi fork().
- Parent ghi chuỗi "Hello from parent" vào đầu ghi của pipe.
- Child đọc dữ liệu từ đầu đọc và in ra màn hình.
- Mỗi process đóng đầu pipe mà mình không sử dụng.

Trả lời:

- p[0] dùng để làm gì?
- p[1] dùng để làm gì?
- Vì sao pipe phải được tạo trước fork()?

Bài 7. Tổng hợp – xây dựng pipeline ls | sort -r

Viết chương trình C tương đương lệnh shell:

```
ls | sort -r
```

Chương trình phải sử dụng:

```
pipe()   fork()   dup2()   close()   execlp()
```

Yêu cầu:

- Child chạy ls và đưa stdout vào đầu ghi của pipe.
- Parent đọc từ đầu đọc của pipe và chạy sort -r qua stdin.
- Đóng các file descriptor không còn sử dụng.
- Không dùng system().

Sau khi hoàn thành, giải thích ngắn vai trò của từng hàm:

```
pipe()  : ......................................
fork()  : ......................................
dup2()  : ......................................
exec()  : ......................................
close() : ......................................
```